Proyecto: "CiberGuardianes" — Gestor de Control y Seguridad en Juegos Infantiles

1. Fundamentación del Dominio

Problema Concreto

En la actualidad, los niños interactúan en plataformas de juego en línea (tipo *Roblox*, *Minecraft* o *Brawl Stars*) exponiéndose a riesgos como compras no autorizadas, contacto con desconocidos o contenido no apto. Los padres y educadores necesitan una herramienta para catalogar, evaluar y gestionar el nivel de seguridad y los permisos de cada juego.

Adaptación del Modelo Base

Se extiende el Registro genérico transformándolo en la entidad **JuegoSeguro**:

Atributo Base	**Atributo Adaptado**	**Tipo**	**Descripción / Regla**
id	id	String	Identificador único (ej: "GAME-001").
nombre	titulo	String	Nombre del videojuego (ej: *"Roblox"*).
categoria	plataforma	Enum	MOBILE, PC, CONSOLA, WEB.
estado	nivelRiesgo	Enum	SEGURO, PRECAUCION, BLOQUEADO.
Nuevo	edadMinima	int	Edad mínima recomendada (ej: 7, 12).
Nuevo	tieneChatAbierto	boolean	Indica si permite chat sin filtros.
Nuevo	comprasIntegradas	boolean	Indica si contiene micropagos/microtransacciones.

Reglas de Negocio Propias

1. **Regla de Riesgo Automático:** Si un juego tiene tieneChatAbierto == true y comprasIntegradas == true, su nivelRiesgo se evalúa como PRECAUCION o BLOQUEADO automáticamente; no puede marcarse como SEGURO.
2. **Regla de Borrado/Protección:** No se puede eliminar del sistema un juego que esté en estado BLOQUEADO a menos que se reautentique la acción (evita modificaciones no autorizadas por parte de un menor).

2. Arquitectura del Sistema (MVC + Fachada + JSON)

Estructura de paquetes recomendada para Java:

Plaintext

```
None
src/
 └── ar.edu.ciberseguridad.juegos/
      ├── model/                      # Entidades y Enums del
Dominio
      │    ├── JuegoSeguro.java
      │    ├── Plataforma.java (Enum)
      │    └── NivelRiesgo.java (Enum)
      ├── repository/                 # Capa de Persistencia
      │    └── JuegoJsonRepository.java
      ├── facade/                     # Fachada de Operaciones
      │    └── CiberSeguridadFacade.java
      ├── controller/                 # Controladores MVC
      │    └── JuegoController.java
      └── view/                       # Interfaz Gráfica Swing
           ├── VentanaPrincipal.java
           ├── FormularioJuegoDialog.java
           └── JuegoTableModel.java
```

3. Implementación de Claves Técnicas

A. Entidad de Dominio (JuegoSeguro.java)

Demuestra encapsulamiento y validación de atributos.

Java

```
None
package ar.edu.ciberseguridad.juegos.model;

public class JuegoSeguro {
    private String id;
    private String titulo;
    private Plataforma plataforma;
```

```
    private NivelRiesgo nivelRiesgo;
    private int edadMinima;
    private boolean tieneChatAbierto;
    private boolean comprasIntegradas;

    public JuegoSeguro(String id, String titulo, Plataforma
plataforma, NivelRiesgo nivelRiesgo,
                       int edadMinima, boolean tieneChatAbierto,
boolean comprasIntegradas) {
        if (titulo == null || titulo.isBlank()) {
            throw new IllegalArgumentException("El título no
puede estar vacío.");
        }
        if (edadMinima < 0 || edadMinima > 18) {
            throw new IllegalArgumentException("La edad mínima
debe estar entre 0 y 18 años.");
        }
        this.id = id;
        this.titulo = titulo;
        this.plataforma = plataforma;
        this.tieneChatAbierto = tieneChatAbierto;
        this.comprasIntegradas = comprasIntegradas;
        this.edadMinima = edadMinima;
        this.nivelRiesgo = evaluarRiesgo(nivelRiesgo);
    }

    // Regla de negocio en el modelo
    private NivelRiesgo evaluarRiesgo(NivelRiesgo sugerido) {
        if (tieneChatAbierto && comprasIntegradas && sugerido ==
NivelRiesgo.SEGURO) {
            return NivelRiesgo.PRECAUCION; // Fuerza nivel de
precaución por seguridad
        }
        return sugerido;
    }

    // Getters y Setters con encapsulamiento...
    public String getId() { return id; }
    public String getTitulo() { return titulo; }
```

```
    public Plataforma getPlataforma() { return plataforma; }
    public NivelRiesgo getNivelRiesgo() { return nivelRiesgo; }
    public int getEdadMinima() { return edadMinima; }
    public boolean isTieneChatAbierto() { return
tieneChatAbierto; }
    public boolean isComprasIntegradas() { return
comprasIntegradas; }

    public void setNivelRiesgo(NivelRiesgo nivelRiesgo) {
        this.nivelRiesgo = evaluarRiesgo(nivelRiesgo);
    }
}
```

B. Fachada (CiberSeguridadFacade.java)

Proporciona acceso unificado a las operaciones del sistema, aislando a Swing de la persistencia y la lógica.

Java

```
None
package ar.edu.ciberseguridad.juegos.facade;

import ar.edu.ciberseguridad.juegos.model.*;
import
ar.edu.ciberseguridad.juegos.repository.JuegoJsonRepository;
import java.util.List;

public class CiberSeguridadFacade {
    private final JuegoJsonRepository repository;

    public CiberSeguridadFacade(JuegoJsonRepository repository) {
        this.repository = repository;
    }

    public List<JuegoSeguro> obtenerTodosLosJuegos() {
        return repository.cargarDesdeJson();
    }

    public void registrarJuego(JuegoSeguro juego) throws
Exception {
        if (repository.existePorId(juego.getId())) {
```

```
            throw new Exception("Ya existe un juego registrado
con el ID: " + juego.getId());
        }
        repository.guardar(juego);
    }

    public void actualizarJuego(JuegoSeguro juego) throws
Exception {
        if (!repository.existePorId(juego.getId())) {
            throw new Exception("El juego a actualizar no
existe.");
        }
        repository.actualizar(juego);
    }

    public void eliminarJuego(String id) throws Exception {
        JuegoSeguro juego = repository.buscarPorId(id);
        if (juego != null && juego.getNivelRiesgo() ==
NivelRiesgo.BLOQUEADO) {
            throw new Exception("No se puede eliminar un juego
con estado BLOQUEADO directamente por protección de seguridad.");
        }
        repository.eliminar(id);
    }
}
```

C. Adaptador de Tabla para Swing (JuegoTableModel.java)

Conecta la lista de objetos JuegoSeguro con el componente JTable de Swing.

Java

```
None
package ar.edu.ciberseguridad.juegos.view;

import ar.edu.ciberseguridad.juegos.model.JuegoSeguro;
import javax.swing.table.AbstractTableModel;
import java.util.List;

public class JuegoTableModel extends AbstractTableModel {
```

```
    private final String[] columnas = {"ID", "Título",
"Plataforma", "Edad Mín.", "Chat Abierto", "Compras", "Nivel
Riesgo"};
    private List<JuegoSeguro> juegos;

    public JuegoTableModel(List<JuegoSeguro> juegos) {
        this.juegos = juegos;
    }

    public void setJuegos(List<JuegoSeguro> juegos) {
        this.juegos = juegos;
        fireTableDataChanged();
    }

    @Override
    public int getRowCount() { return juegos.size(); }

    @Override
    public int getColumnCount() { return columnas.length; }

    @Override
    public String getColumnName(int col) { return columnas[col];
}

    @Override
    public Object getValueAt(int rowIndex, int columnIndex) {
        JuegoSeguro j = juegos.get(rowIndex);
        switch (columnIndex) {
            case 0: return j.getId();
            case 1: return j.getTitulo();
            case 2: return j.getPlataforma();
            case 3: return j.getEdadMinima() + " años";
            case 4: return j.isTieneChatAbierto() ? "Sí ⚠️" :
"No";
            case 5: return j.isComprasIntegradas() ? "Sí 💳" :
"No";
            case 6: return j.getNivelRiesgo();
            default: return null;
        }
```

```
    }

    public JuegoSeguro getJuegoAt(int row) {
        return juegos.get(row);
    }
}
```

4. Guía para la Defensa Expositiva del Proyecto

Al momento de defender el trabajo ante el tribunal docente, utilicen los siguientes argumentos clave:

- **¿Cómo fundamentaron la elección del dominio?**
  "Convertimos el registro genérico en un catálogo de supervisión parental de ciberseguridad infantil. Agregamos reglas reales de evaluación de riesgos basadas en factores como chat abierto y compras dentro de la app."
- **¿Por qué la Fachada y no llamadas directas desde los botones de Swing?**
  "Aplicamos la Fachada para desacoplar completamente la vista (Swing) de las reglas de negocio y de la persistencia JSON. Esto permite que el sistema sea testeable y facilita cambiar Swing por otra interfaz sin tocar la lógica."
- **¿Cómo garantizan la persistencia?**
  "Toda operación de modificación (Alta, Baja, Edición) ejecutada mediante la Fachada se delega al repositorio JSON, reescribiendo el estado actualizado de la colección para asegurar que los datos no se pierdan entre ejecuciones."